Case Study 74

EventSphere - Conference and Event Registration System

Name: Mohammed Saad Mulla

Roll No: 150096725112

Course: B.Tech, 2025-2029

Cohort: Sam Altman

1. What this project is

EventSphere is a database for managing multi session conferences. Attendees register for individual sessions inside an event. The core rule is simple: the same attendee can never be registered for the same session twice. Organisers also need to be able to see, at any time, how many people are registered for each session, so rooms and catering can be planned properly.

This document is a full working record of everything built and discussed for this project: the entities, the schema, the normalisation fix, every query written, the ER diagram logic, and all the underlying DBMS theory that supports it.

2. Entities and relationships

There are five entities in this system:

- Event - the overall conference, for example TechConf 2026
- Session - one talk or slot inside an event, for example AI Ethics Panel at 10am
- Speaker - the person presenting a session
- Attendee - the person registering
- Registration - the link between an attendee and a session

How they relate to each other:

- One Event has many Sessions -> this is a 1 to M relationship
- One Speaker can present many Sessions -> also 1 to M
- One Attendee can register for many Sessions, and one Session can have many Attendees -> this is Many to Many (M:N)

A real database table can never store M:N directly. A foreign key column can only point to one row at a time, it cannot point to many rows at once. So the M:N relationship between Attendee and Session is broken down into two clean 1:M relationships, using Registration as a bridge (junction) table in the middle.

- Attendee -> Registration is 1:M (one attendee has many registration rows)
- Session -> Registration is 1:M (one session has many registration rows)

Put together, these two 1:M relationships recreate the effect of M:N, without ever needing to store it directly. This is the standard way any M:N relationship is handled in a relational database.

3. The normalisation fix (2NF)

The original brief pointed out a design flaw on purpose: speaker name and bio were being repeated on every single session row, which wastes space and risks the same speaker's data going out of sync across rows. The fix is to pull Speaker into its own separate table, linked back to Session using speaker_id as a foreign key. Now speaker information is stored exactly once, no matter how many sessions that speaker is doing. This is a 2NF (Second Normal Form) fix, removing a repeating, non key dependent attribute into its own table.

4. Full schema (table by table)

events

```
event_id      INT, PRIMARY KEY
event_name    VARCHAR(100), NOT NULL
event_date    DATE, NOT NULL
venue         VARCHAR(100), NOT NULL
```

speakers

```
speaker_id    INT, PRIMARY KEY
speaker_name  VARCHAR(100), NOT NULL
bio           VARCHAR(300)
```

sessions

```
session_id    INT, PRIMARY KEY
event_id      INT, FOREIGN KEY -> events(event_id)
speaker_id    INT, FOREIGN KEY -> speakers(speaker_id)
session_name  VARCHAR(150), NOT NULL
session_date  DATE, NOT NULL
start_time    TIME, NOT NULL
end_time      TIME, NOT NULL
room          VARCHAR(50), NOT NULL
```

attendees

```
attendee_id   INT, PRIMARY KEY
attendee_name VARCHAR(100), NOT NULL
email         VARCHAR(100), NOT NULL
```

```
phone        VARCHAR(20)
```

registrations

```
registration_id    INT, PRIMARY KEY
attendee_id        INT, FOREIGN KEY -> attendees(attendee_id)
session_id         INT, FOREIGN KEY -> sessions(session_id)
registration_time   DATETIME, NOT NULL
CONSTRAINT uq_attendee_session UNIQUE (attendee_id, session_id)
```

The UNIQUE constraint on (attendee_id, session_id) in registrations is the advanced concept the brief specifically asked for. It makes it physically impossible for the same attendee and session pair to appear twice in that table. If someone tries to insert a duplicate pair, the database itself rejects it automatically, there is no need to check for duplicates manually in code.

5. Seed data summary

- 3 events: TechConf 2026, AI Summit 2026, DevFest 2026
- 6 speakers
- 8 sessions spread across the 3 events
- 10 attendees
- 30 registrations in total, with no repeated (attendee_id, session_id) pair

Registrations per session work out to 5, 4, 4, 5, 3, 3, 3, 3 for sessions 1 to 8, adding up to 30. One attendee, Aarav Shah (attendee_id 1), was deliberately given 4 registrations so that a later query filtering for attendees with more than 3 registrations actually returns a row during a live demo, instead of coming back empty.

6. All queries written, with explanation

Query 1 (the one specifically required by the brief)

Count how many attendees are registered for each session.

```
SELECT se.session_id,
    se.session_name,
    COUNT(r.registration_id) AS attendee_count
FROM sessions se
LEFT JOIN registrations r
  ON se.session_id = r.session_id
GROUP BY se.session_id, se.session_name
ORDER BY se.session_id;
```

LEFT JOIN is used on purpose, so that a session with zero registrations would still show up in the result with a count of 0, instead of disappearing entirely.

Query 2: speaker schedule

Every session a given speaker is presenting, across the whole conference.

```
SELECT sp.speaker_name,
    se.session_name,
    se.session_date,
    se.start_time,
    se.end_time,
    se.room
FROM speakers sp
JOIN sessions se
  ON sp.speaker_id = se.speaker_id
ORDER BY sp.speaker_name, se.session_date, se.start_time;
```

Query 3: full registration detail

Attendee, session and event in one row, joined together. This also demonstrates referential integrity, since every registration is guaranteed to trace back to a real attendee, session and event because of the foreign keys.

```
SELECT r.registration_id,
    a.attendee_name,
    ev.event_name,
    se.session_name,
    se.session_date,
    r.registration_time
FROM registrations r
JOIN attendees a ON r.attendee_id = a.attendee_id
JOIN sessions se ON r.session_id = se.session_id
JOIN events ev ON se.event_id = ev.event_id
ORDER BY ev.event_name, se.session_name, r.registration_time;
```

Query 4: total registrations per event

Useful for catering and room planning at the whole event level, not just per session.

```
SELECT ev.event_name,
    COUNT(r.registration_id) AS total_registrations
FROM events ev
JOIN sessions se ON ev.event_id = se.event_id
LEFT JOIN registrations r ON se.session_id = r.session_id
GROUP BY ev.event_id, ev.event_name
ORDER BY total_registrations DESC;
```

Query 5: busy attendees

Attendees registered for more than 3 sessions. This uses HAVING instead of WHERE, because the filter condition depends on COUNT, which only exists after the rows have already been grouped.

```
SELECT a.attendee_name,
    COUNT(r.registration_id) AS sessions_registered
FROM attendees a
JOIN registrations r ON a.attendee_id = r.attendee_id
GROUP BY a.attendee_id, a.attendee_name
HAVING COUNT(r.registration_id) > 3
ORDER BY sessions_registered DESC;
```

Demonstration: the UNIQUE constraint actually working

Attendee 1 is already registered for session 1. Running the insert below is meant to fail on purpose, in front of judges if needed, to prove the constraint works.

```
INSERT INTO registrations (registration_id, attendee_id, session_id, registration_time)
VALUES (31, 1, 1, '2026-11-05 09:00:00');
```

Note: the registration_id used here must be a fresh, unused id. Using an id that already exists (such as 30, which is already taken by the seed data) would cause MySQL to fail on the PRIMARY KEY check first, which would demonstrate the wrong thing. Id 31 is unused, so the only thing that can fail is the UNIQUE pair check, giving the correct error:

```
ERROR 1062 (23000): Duplicate entry '1-1' for key 'registrations.uq_attendee_session'
```

7. ER diagram logic

The ER diagram has five boxes: Events, Speakers, Sessions, Attendees, Registrations. Every single connector line on the diagram is labelled 1:M, there is no line anywhere labelled M:N.

This is intentional, not a mistake. As explained above, Attendee and Session are conceptually Many to Many, but that can never be drawn as a direct connection in a real relational schema. Instead, Registration sits in the middle as a bridge table. Attendee connects to Registration as 1:M, and Session connects to Registration as 1:M. The two 1:M relationships together recreate M:N behaviour without ever storing it directly.

If asked directly where the M:N relationship is, the answer is: it was resolved through the Registration bridge table, which turns one M:N relationship into two clean 1:M relationships, because a relational database cannot store M:N directly.

8. Core DBMS theory used in this project

Keys

- Candidate key - any column that could uniquely identify a row on its own
- Primary key (PK) - the one candidate key actually chosen, must be unique and never empty
- Foreign key (FK) - a column that points to another table's primary key, linking two tables together
- UNIQUE constraint - forces a column, or a combination of columns, to never repeat

SQL statement categories

- DDL (Data Definition Language) - CREATE, ALTER, DROP, defines the structure of tables
- DML (Data Manipulation Language) - INSERT, UPDATE, DELETE, changes the actual data
- DQL (Data Query Language) - SELECT, reads data without changing it

Joins

- JOIN - temporarily combines two tables together based on a matching column
- ON - the exact rule used to decide which rows from each table match each other
- Table alias (for example se for sessions) - a short nickname for a table, used to avoid repeating long names and to separate columns that share the same name across tables
- INNER JOIN - only returns rows that have a match in both tables

7.1 ER Diagram

The diagram below shows the five tables of EventSphere and how they connect. Every connector is **1:M**; the Attendee-to-Session many-to-many relationship is resolved through the **Registrations** bridge table.

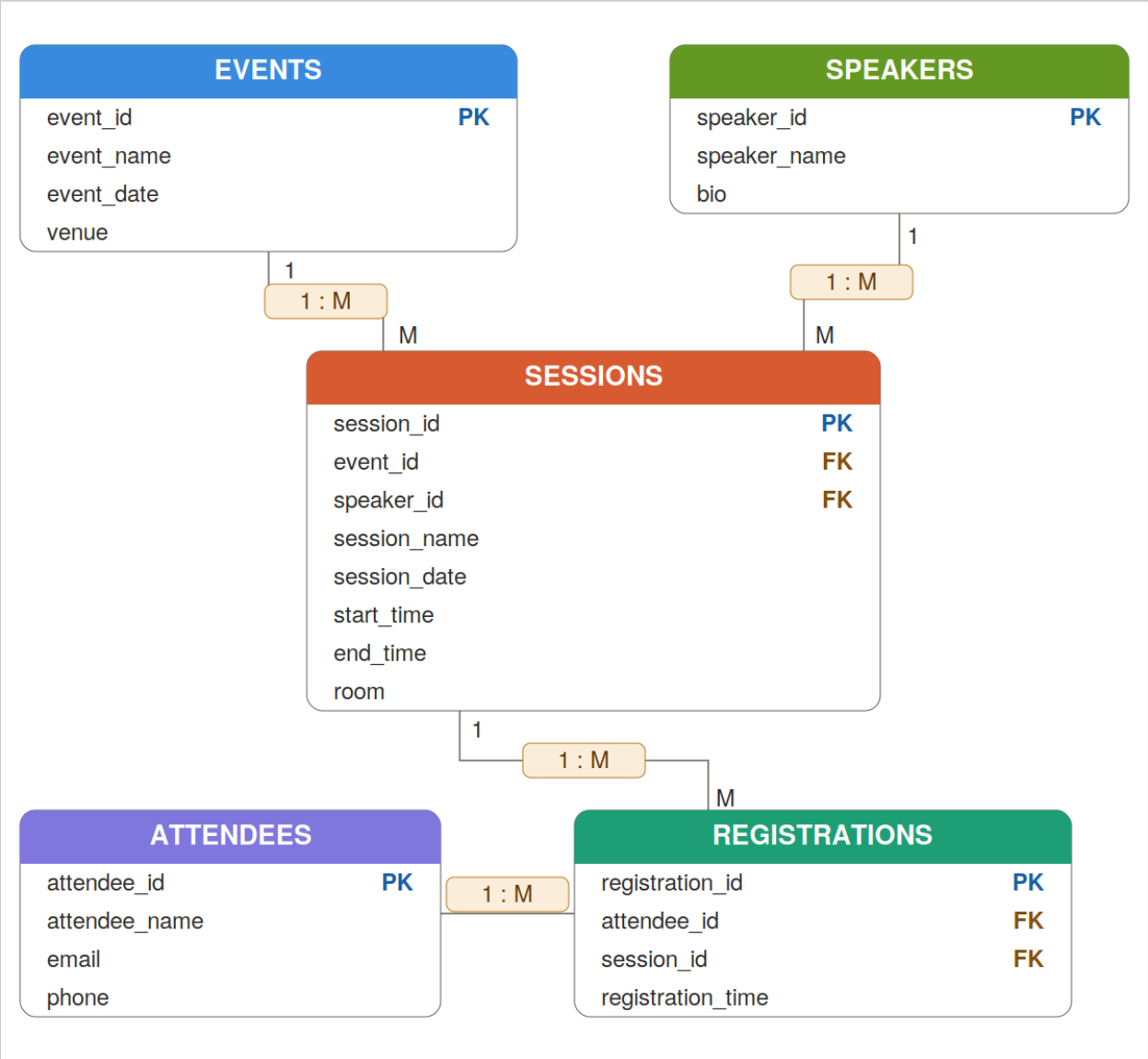


Figure 1: EventSphere entity relationship diagram

How to read it

- PK - primary key; FK - foreign key pointing to another table's primary key
- 1 : M - one row on the "1" side relates to many rows on the "M" side
- Registrations holds attendee_id and session_id as foreign keys, with a UNIQUE pair

- LEFT JOIN - returns everything from the left (first) table, even rows with no match on the right, filling missing values with NULL
- RIGHT JOIN - the mirror of LEFT JOIN, keeps everything from the right table instead
- FULL JOIN - keeps everything from both tables, matched or not
- Self join - a table joined to itself, used for example to compare rows within the same table against each other

Grouping and aggregation

- GROUP BY - groups rows into buckets so aggregate functions work per group instead of across the whole table at once
- COUNT, SUM, AVG, MAX, MIN - the common aggregate functions used with GROUP BY
- HAVING - filters groups after aggregation has already happened, unlike WHERE which filters rows before any grouping
- COALESCE - replaces a NULL result (for example SUM on zero rows) with a fallback value such as 0, so later calculations do not break

Subqueries

- Scalar subquery - an inner query that returns exactly one single value, and runs only once
- Correlated subquery - an inner query that depends on the outer row, so it has to run again separately for every single outer row
- IN - checks a value against a list of possible values
- NOT EXISTS - checks whether there are zero matching rows for something
- ALL - compares a value against every single row returned by a subquery, not just one

ACID properties (reliable transactions)

- Atomicity - every step in a transaction either all succeed together, or none of them do
- Consistency - the data stays logically correct before and after any transaction
- Isolation - transactions running at the same time do not interfere with each other
- Durability - once something is saved, it stays saved even if the system crashes right after

Database design concepts

- Normalisation - organising tables to reduce repeated or redundant data, done in stages (1NF, 2NF, 3NF, and so on)
- Referential integrity - foreign keys guarantee that every row always points to something that genuinely exists
- Entity - a real world thing the database stores information about, for example Attendee or Session
- Attribute - a property or column describing an entity
- Cardinality - how many rows on one side of a relationship can connect to how many rows on the other side (1:1, 1:M, M:N)
- Bridge or junction table - a table used to break a true M:N relationship down into two clean 1:M relationships

9. Where things stand

The schema, the seed data, all required and supporting queries, and the ER diagram logic are complete and have been checked by hand for correctness, including fixing a bug where the live UNIQUE constraint demo would originally have failed on the wrong error (a primary key clash instead of the actual unique constraint clash), which has since been corrected to use a fresh id.

What is still pending is redrawing the ER diagram independently (for example in draw.io) as a personal exercise, and final rehearsal of explaining the schema, the queries, and the M:N to bridge table resolution out loud, since this will be presented live in front of judges who may ask about any part of it.